

Improved Proposed Wording for LWG 2114

- [Introduction](#)
- [Revision History](#)
- [Discussion](#)
- [Rationale](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Acknowledgements](#)
- [Bibliography](#)

Introduction

This proposal is intended to provide wording to resolve the existing library issue [LWG 2114](#).

Revision History

(Initial version)

Discussion

Issue [LWG 2114](#) has already a long history and a number of wording revisions that went backward and forward. But with the introduction of the *boolean-testable* exposition-only concept by [P1964R2](#) adopted during the Prague 2020 meeting we have now an officially accepted useful tool that can be used to fix this issue.

Since the edits of the working draft are not very small, this paper has been written.

Rationale

The *boolean-testable* has all the properties that [LWG 2114](#) tried to specify without having language-concepts available. Our existing terminology of "modeling" a concept should allow us to use this wording even when we specify something within existing named requirements sets. So the approach of this paper is just to impose the *boolean-testable* concept requirements to those places that the issue identifies, now updated to the most recent working draft.

The following guidelines have been chosen to decide for applying the *boolean-testable* concept requirements below:

- Type families that are potentially influenced by user-defined types, such as types meeting the *Cpp17NullablePointer* requirements, are usually good candidates, unless the `bool`-like expressions are purposely intended to reflect the exact behaviour as presented (The design of the comparison functions of `optional` belong into this category)

- Type families or specifications that solely exist to shorten the specification of library-provided types, should usually *not* be good candidates, unless the intention exists here to provide Library implementors more freedom. An example for this case are the Container requirements which are actually invented to shorten the specification of the existing concrete container templates, where all of them use a concrete type `bool` for `operator==/!=` and for the empty function

It should be pointed out that in the past we had considered to take advantage of "convertible to `bool`" requirements for allocator equality by allowing to implement them as `constexpr` with `std::true/false_type` as return types, but with the introduction of the `is_always_equal` query type of an allocator we had withdrawn this idea, so at the moment we have no need for such a customization point. If we really want to return to a similar point again, we can easily decide to change the return type e.g. for container equality again to model *boolean-testable*, but now specifically to realize a concrete purpose.

Resolved Issues

If the proposed resolution will be accepted, the following library issues will be resolved:

Number	Description
LWG 2114	Incorrect " <i>contextually</i> convertible to <code>bool</code> " requirements

Proposed resolution

The proposed wording changes refer to [N4861](#).

1. Change Table [tab:cpp17.equalitycomparable] as indicated:

Table 25: *Cpp17EqualityComparable* requirements [tab:cpp17.equalitycomparable]

Expression	Return type	Requirement
<code>a == b</code>	convertible to <code>bool</code> <code>decltype(a == b)</code> models <i>boolean-testable</i>	<code>==</code> is an equivalence relation, that is, it has the following properties: [...]

2. Change Table [tab:cpp17.less-than-comparable] as indicated:

Table 26: *Cpp17LessThanComparable* requirements [tab:cpp17.less-than-comparable]

Expression	Return type	Requirement
<code>a < b</code>	convertible to <code>bool</code> <code>decltype(a < b)</code> models <i>boolean-testable</i>	<code><</code> is a strict weak ordering relation (25.8 [alg.sorting])

3. Change in [nullable-pointer.requirements] Table [tab:cpp17.nullable-pointer] as indicated:

Table 33: *Cpp17NullablePointer* requirements [tab:cpp17.nullable-pointer]

Expression	Return type	Operational semantics
...		
<code>a != b</code>	contextually convertible to <code>bool</code> <code>decltype(a != b)</code> models <i>boolean-testable</i>	<code>!(a == b)</code>

a == np np == a	contextually convertible to bool decltype(a == np) and decltype(np == a). <u>both model <i>boolean-testable</i></u>	a == P()
a != np np != a	contextually convertible to bool decltype(a != np) and decltype(np != a). <u>both model <i>boolean-testable</i></u>	!(a == np)

4. Change 17.11.6 [cmp.alg] as indicated:

-4- The name `compare_strong_order_fallback` denotes a customization point object (16.4.2.2.6). Given subexpressions `E` and `F`, the expression `compare_strong_order_fallback(E, F)` is expression-equivalent (16.3.11) to:

[...]

(4.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and ~~convertible to bool~~ the types `decltype(E == F)` and `decltype(E < F)` both model *boolean-testable*,

```
E == F ? strong_ordering::equal :
E < F  ? strong_ordering::less  :
        strong_ordering::greater
```

except that `E` and `F` are evaluated only once.

[...]

[...]

-5- The name `compare_weak_order_fallback` denotes a customization point object (16.4.2.2.6). Given subexpressions `E` and `F`, the expression `compare_weak_order_fallback(E, F)` is expression-equivalent (16.3.11) to:

[...]

(5.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and ~~convertible to bool~~ the types `decltype(E == F)` and `decltype(E < F)` both model *boolean-testable*,

```
E == F ? weak_ordering::equivalent :
E < F  ? weak_ordering::less      :
        weak_ordering::greater
```

except that `E` and `F` are evaluated only once.

[...]

[...]

-6- The name `compare_partial_order_fallback` denotes a customization point object (16.4.2.2.6). Given subexpressions `E` and `F`, the expression `compare_partial_order_fallback(E, F)` is expression-equivalent (16.3.11) to:

[...]

(6.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and ~~convertible to bool~~ the types `decltype(E == F)` and `decltype(E < F)` both model

boolean-testable,

```
E == F ? partial_ordering::equivalent :
E < F  ? partial_ordering::less      :
E > F  ? partial_ordering::greater   :
        partial_ordering::unordered
```

except that E and F are evaluated only once.

[...]

5. Change 20.5.8 [tuple.rel] as indicated:

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

-1- *Mandates*: For all i, where $0 \leq i < \text{sizeof}...(TTypes)$, $\text{get}<i>(t) == \text{get}<i>(u)$ is a valid expression ~~returning a type that is convertible to bool~~ and the type `decltype(get<i>(t) == get<i>(u))` models *boolean-testable*.
`sizeof...(TTypes)` equals `sizeof...(UTypes)`.

6. Keep 20.6.6 [optional.relops] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
7. Keep 20.6.8 [optional.comp.with.t] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
8. Keep 20.7.6 [variant.relops] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
9. Keep 20.15.8 [meta.logical] unchanged: These logical type traits do already the boolean logic for you, and no further requirements are imposed.

10. Change Table [tab:container.req] as indicated:

[Drafting note: Given that the containers are no user-provided types it seems like an unnecessary generic allowance to support types that are convertible to bool below. In particular, because the container requirements tables would then be in (weak) conflict with the header and class synopses of the corresponding containers, which all specify concrete bool for these operations.

For this specific wording change here, the author has a mild preference to get rid of the "convertibility" freedom. The alternative would be declare these types as "modeling *boolean-testable*".]

Table 73: Container requirements [tab:container.req]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
...				
a == b	convertible to bool	== is an equivalence relation. equal(a.begin(),	[...]	[...]

		a.end(), b.begin(), b.end())		
a != b	convertible to bool	Equivalent to !(a == b)	[...]	[...]
...				
a.empty()	convertible to bool	a.begin() == a.end()	[...]	[...]

11. Keep 23.3.4.4 [iterator.concept.winc] unchanged: It seems to me that sufficient wording exists to exclude funny integer-class types, and they are all under control by the implementation.

12. Change in [iterator.cpp17] Table [tab:inputiterator] and Table [tab:randomaccessiterator] as indicated:

Table 85: *Cpp17InputIterator* requirements (in addition to *Cpp17Iterator*)
[tab:inputiterator]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
a != b	contextually convertible to bool decltype(a != b). models <i>boolean-testable</i>	!(a == b)	[...]
...			

[...]

Table 89: *Cpp17RandomAccessIterator* requirements (in addition to *Cpp17BidirectionalIterator*) [tab:randomaccessiterator]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
a < b	contextually convertible to bool decltype(a < b). models <i>boolean-testable</i>	b - a > 0	[...]
a > b	contextually convertible to bool decltype(a > b). models <i>boolean-testable</i>	b < a	[...]
a >= b	contextually convertible to bool decltype(a >= b). models <i>boolean-testable</i>	!(a < b)	[...]
a <= b	contextually convertible to bool decltype(a <= b). models <i>boolean-testable</i>	!(a > b)	[...]

13. Keep 23.5.1.7 [reverse.iter.cmp] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.

14. Keep 23.5.3.7 [move.iter.op.comp] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.

15. Change 25.2 [algorithms.requirements] as indicated:

[Drafting note: The wording changes below also fix (a) unusual wording forms used ("should work") which are unclear in which sense they are imposing normative requirements and (b) the problem, that the current wording seems to allow that the predicate may mutate a call argument, if that is not a dereferenced iterator. Upon applying the new wording it became obvious that the both the previous and the new wording has the effect that currently algorithms such as `adjacent_find`, `search_n`, `unique`, and `unique_copy` are not correctly described (because they have no iterator argument named `first1`), which could give raise to a new library issue.]

-7- When not otherwise constrained, the `Predicate` parameter is used whenever an algorithm expects a function object (20.14 [function.objects]) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as true. ~~In other words, if an algorithm takes `Predicate` `pred` as its argument and `first` as its iterator argument with value type `T`, it should work correctly in the construct `pred(*first)` contextually converted to `bool` (7.3 [conv])~~ the expression `pred(*first)` shall be well-formed and the type `decltype(pred(*first))` shall model *boolean-testable* (18.5.2 [concept.boolean-testable]). The function object `pred` shall not apply any non-constant function through ~~the dereferenced iterator~~ its argument. Given a glvalue `u` of type (possibly const) `T` that designates the same object as `*first`, `pred(u)` shall be a valid expression that is equal to `pred(*first)`.

-8- When not otherwise constrained, the `BinaryPredicate` parameter is used whenever an algorithm expects a function object that when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type `T` when `T` is part of the signature returns a value testable as true. ~~In other words, if an algorithm takes `BinaryPredicate` `binary_pred` as its argument and `first1` and `first2` as its iterator arguments with respective value types `T1` and `T2`, it should work correctly in the construct `binary_pred(*first1, *first2)` contextually converted to `bool` (7.3 [conv])~~ the expression `binary_pred(*first1, *first2)` shall be well-formed and the type `decltype(binary_pred(*first1, *first2))` shall model *boolean-testable*. Unless otherwise specified, `BinaryPredicate` always takes the first iterator's value_type as its first argument, that is, in those cases when `T` value is part of the signature, ~~it should work correctly in the construct `binary_pred(*first1, value)` contextually converted to `bool` (7.3 [conv])~~ the expression `binary_pred(*first1, value)` shall be well-formed and the type `decltype(binary_pred(*first1, value))` shall model *boolean-testable*. `binary_pred` shall not apply any non-constant function through ~~the dereferenced iterators~~ any of its arguments. Given a glvalue `u` of type (possibly const) `T1` that designates the same object as `*first1`, and a glvalue `v` of type (possibly const) `T2` that designates the same object as `*first2`, `binary_pred(u, *first2)`, `binary_pred(*first1, v)`, and `binary_pred(u, v)` shall each be a valid expression that is equal to `binary_pred(*first1, *first2)`, and `binary_pred(u, value)` shall be a valid expression that is equal to `binary_pred(*first1, value)`.

16. Change 25.8 [alg.sorting] as indicated:

[Drafting note: The existing wording inherits all the good wording from `BinaryPredicate`, that we fixed above, so there is only little to do but specifying the conversion to `bool` less strict, since we already know that it is a type that models *boolean-testable*]

-2- `Compare` is a function object type (20.14 [function.objects]) that meets the requirements for a template parameter named `BinaryPredicate` (25.2 [algorithms.requirements]). The return value of the function call operation applied to an object of type `Compare`, when ~~contextually converted to `bool` (7.3 [conv])~~, yields true if the first argument of the call is less than the

second, and false otherwise. Compare comp is used throughout for algorithms assuming an ordering relation.

17. Change 26.7.3.2 [valarray.comparison] as indicated:

```
template<class T> valarray<bool> operator==(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator<=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator|| (const valarray<T>&, const valarray<T>&);
```

Let @ denote the indicated binary operator, and let E be the expression declval<const T>(). @ declval<const T>().

~~-1- Mandates: The indicated operator can be applied to operands of type T and returns a value of type bool or which can be unambiguously implicitly converted to bool.~~
The expression E is well-formed and the type decltype(E) models boolean-testable (18.5.2 [concept.booleantestable]).

[...]

```
template<class T> valarray<bool> operator==(const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator==(const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator!=(const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator< (const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator> (const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator<=(const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator<=(const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator>=(const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator&&(const typename valarray<T>::value_type&,
                                             const valarray<T>&);
template<class T> valarray<bool> operator|| (const valarray<T>&,
                                             const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator|| (const typename valarray<T>::value_type&,
                                             const valarray<T>&);
```

Let @ denote the indicated binary operator, and let E be the expression declval<const T>(). @ declval<const T>().

~~-4- Mandates: The indicated operator can be applied to operands of type T and returns a value of type bool or which can be unambiguously implicitly converted~~

~~to bool~~ The expression E is well-formed and the type `decltype(E)` models *boolean-testable* (18.5.2 [concept.booleantestable]).

[...]

18. Change in 29.5.4.2 [fpos.operations] Table [tab:fpos.operations] as indicated:

Table 121: Position type requirements [tab:fpos.operations]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
<code>p != q</code>	convertible to bool <code>decltype(p != q)</code> models <i>boolean-testable</i>	<code>!(p == q)</code>	
...			

19. Change 32.2.1 [thread.req.paramname] as indicated:

[Drafting note: The following performs some minor drive-by fixes to fix minor wording issues that would e.g. exclude normal function pointers to be used as predicate. Note that we intentionally do *not* describe a const lvalue `pred`, since there is nothing in the specification that would imply or require that.]

-1- Throughout this Clause, the names of template parameters are used to express type requirements. ~~If a template parameter is named `Predicate`, `operator(.)` applied to the template argument shall return a value that is convertible to bool~~ `Predicate` is a function object type (20.14 [function.objects]). Let `pred` denote an lvalue of type `Predicate`. Then the expression `pred(.)` shall be well-formed and the type `decltype(pred(.))` shall model *boolean-testable* (18.5.2 [concept.booleantestable]). The return value of `pred(.)`, converted to `bool`, yields `true` if the corresponding test condition is satisfied, and `false` otherwise. [...]

Acknowledgements

Thanks to Barry Revzin and Tim Song for reviewing this proposal and providing helpful improvement suggestions.

Bibliography

[N4861] Richard Smith: "Working Draft, Standard for Programming Language C++", 2020
<https://wg21.link/n4861>

[P1964R2] Tim Song: "Wording for *boolean-testable*", 2020
<https://wg21.link/p1964r2>

[LWG2114] Daniel Krügler: "Incorrect "*contextually* convertible to `bool`" requirements", 2011
<https://wg21.link/lwg2114>